

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Направление: 01.04.02 Математика и компьютерные науки

ООП: Разработка программного обеспечения и науки о данных

ОТЧЕТ О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

Тема:

Разработка драйвера блочного устройства с автоматической коррекцией ошибок

Выполнил:

Смирнов Алексей Артёмович, магистрант, группа 25.М02-мкн

Руководитель научно–исследовательской работы:

Ловягин Никита Юрьевич, кандидат физико-математических наук, доцент кафедры информатики

САНКТ-ПЕТЕРБУРГ

2026

Содержание

1 Введение **3**

2 Постановка задачи **5**

3 Анализ существующих решений **6**

3.1 RAID–массивы 6

3.2 Файловые системы с контролем целостности (Btrfs, ZFS) 7

3.3 Device Mapper Integrity 7

4 Коды исправления ошибок (ECC) **9**

5 Архитектура драйвера **10**

5.1 Разбиение данных и метаданных на устройстве 10

5.2 Синхронизация данных и метаданных 11

5.3 Поток выполнения операции записи 12

5.4 Поток выполнения операции чтения 16

6 Результаты работы **19**

6.1 Проверка корректности драйвера 19

6.1.1 Проверка корректности операций чтения и записи 19

6.1.2 Проверка работы с файловой системой 19

6.1.3 Проверка обнаружения повреждения данных 20

6.1.4 Итоги проверки корректности 21

6.2 Тесты производительности 21

6.2.1 Методика проведения тестов производительности 21

6.2.2 Результаты тестов производительности 22

6.3 Предложения по улучшению архитектуры 29

7 Заключение **31**

8 Список использованных источников **32**

1. Введение

Современные накопители обеспечивают высокую надёжность хранения данных, однако вероятность возникновения ошибок чтения и записи полностью исключить невозможно. Причинами повреждения данных могут являться деградация носителя, ошибки передачи данных, внезапное отключение питания, аппаратные сбои контроллера или внешние воздействия на устройство хранения. Даже единичное повреждение блока данных способно привести к потере важных файлов или нарушению целостности файловой системы.

Для решения подобных проблем в промышленных системах хранения применяются сложные механизмы обеспечения надёжности: RAID-массивы, журнальные файловые системы, контрольные суммы, технологии самовосстановления данных и специализированные системы хранения. Однако подобные решения ориентированы на серверные и корпоративные сценарии эксплуатации, требуют дополнительного оборудования либо обладают значительной сложностью настройки и сопровождения.

Целью данной работы является разработка драйвера блочного устройства для операционной системы Linux, обеспечивающего дополнительный уровень контроля целостности данных непосредственно на уровне блочного ввода-вывода. Разрабатываемый драйвер предназначен не для построения высокодоступных корпоративных систем хранения, а для использования в персональных сценариях, где требуется повысить надёжность хранения важной информации на отдельном накопителе, например на внешнем USB-диске, архивном носителе или устройстве резервного копирования.

В работе реализован драйвер Device Mapper, создающий виртуальное блочное устройство поверх существующего физического накопителя. Драйвер прозрачно перехватывает операции чтения и записи, выполняет преобразование адресного пространства и хранит служебные метаданные совместно с пользовательскими данными. Для каждого блока данных вычисляется контрольная сумма, которая записывается в специально выделенную область носителя. При последующем чтении выполняется повторное вычисление контрольной суммы и проверка целостности содержимого блока. Более подробное описание архитектуры и принципов работы драйвера приведено в разделе «Архитектура драйвера».

Текущая реализация обеспечивает обнаружение повреждений данных. Разработанная архитектура предусматривает возможность дальнейшего расширения функциональности за счёт интеграции кодов исправления ошибок (ECC), что позволит не только

выявлять нарушения целостности данных, но и восстанавливать повреждённую информацию на уровне блочного устройства.

Весь исходный код драйвера доступен по ссылке [13].

2. Постановка задачи

Целью работы является разработка драйвера блочного устройства для операционной системы Linux, обеспечивающего контроль целостности данных на уровне операций блочного ввода-вывода. Драйвер должен функционировать как промежуточный слой между файловой системой и физическим накопителем, оставаясь полностью прозрачным для пользовательских приложений и не требуя модификации существующих файловых систем.

Разрабатываемый драйвер должен выполнять обработку операций чтения и записи, поступающих от операционной системы, и организовывать хранение служебной информации, необходимой для контроля целостности данных.

При выполнении операций записи драйвер должен вычислять контрольные суммы для сохраняемых данных и записывать их в служебную область накопителя. При последующем чтении необходимо выполнять повторное вычисление контрольной суммы и сравнение полученного значения с ранее сохранённым. В случае обнаружения расхождения драйвер должен фиксировать факт нарушения целостности данных и сообщать о возникновении ошибки. Реализация механизма контроля целостности данных рассматривается как первый этап разработки системы автоматической коррекции ошибок на уровне блочного устройства. Обнаружение повреждений данных является необходимым условием для последующего внедрения алгоритмов восстановления информации на основе кодов исправления ошибок (ECC).

Дополнительным требованием является обеспечение корректной работы в условиях параллельного доступа к устройству. Драйвер должен гарантировать согласованность пользовательских данных и соответствующих им метаданных при одновременном выполнении нескольких операций чтения и записи.

Разрабатываемое решение должно быть реализовано в виде модуля ядра Linux с использованием подсистемы Device Mapper. Полученная реализация должна стать основой для дальнейшего расширения функциональности драйвера, включая поддержку механизмов автоматической коррекции ошибок на основе кодов исправления ошибок (ECC).

3. Анализ существующих решений

Большинство существующих решений реализуют защиту данных либо на уровне массива накопителей (RAID), либо на уровне файловой системы (Btrfs, ZFS). В обоих случаях контроль целостности реализуется выше уровня блочного устройства и зависит от используемой файловой системы либо конфигурации массива накопителей.

В рамках данной работы предлагается иной подход. Разрабатываемый драйвер реализуется на уровне блочного устройства и функционирует как промежуточный слой между файловой системой и физическим накопителем и не требует дополнительных решений и утилит. Благодаря этому обеспечивается независимость от используемой файловой системы и возможность применения существующих файловых систем без модификации их внутренней структуры.

3.1. RAID–массивы

Одним из наиболее распространённых способов повышения надёжности хранения данных являются RAID-массивы (Redundant Array of Independent Disks) [1]. Основная идея технологии RAID заключается в распределении данных между несколькими физическими накопителями с целью повышения производительности, отказоустойчивости либо одновременного достижения обоих эффектов. Различные уровни RAID используют зеркалирование данных, контрольную информацию или их комбинацию.

Наиболее распространёнными являются следующие уровни RAID:

- RAID 1 — зеркалирование данных на два и более накопителя;
- RAID 5 — распределение данных и контрольной информации (паритета) между несколькими дисками;
- RAID 6 — аналог RAID 5 с двойным паритетом;
- RAID 10 — комбинация зеркалирования и чередования данных.

Основным преимуществом RAID является устойчивость к отказу одного или нескольких физических накопителей. Однако классические RAID-системы ориентированы преимущественно на защиту от полного отказа диска и не обеспечивают полноценного контроля целостности данных на уровне отдельных блоков. В случае так называемого «тихого повреждения данных» (silent data corruption), когда накопитель возвращает некорректные данные без сообщения об ошибке ввода-вывода, RAID не всегда

способен определить, какая из копий данных является корректной. Поэтому современные системы хранения обычно дополняют RAID механизмами контроля целостности на более высоких уровнях программного стека.

Таким образом, RAID решает задачу отказоустойчивости накопителей, но не является полноценным средством обнаружения и исправления повреждений данных внутри отдельных блоков.

3.2. Файловые системы с контролем целостности (Btrfs, ZFS)

Современные файловые системы нового поколения, такие как Btrfs и ZFS, реализуют концепцию сквозного контроля целостности данных (end-to-end integrity).

Файловая система Btrfs [2] вычисляет контрольные суммы для пользовательских данных и метаданных при записи и проверяет их при чтении. Контрольные суммы хранятся отдельно от самих данных, что позволяет обнаруживать повреждения блоков независимо от файловой системы или накопителя. Кроме того, при использовании RAID-профилей Btrfs способна автоматически восстанавливать повреждённые блоки из избыточных копий данных.

Файловая система ZFS [3] использует аналогичный подход. Для каждого блока вычисляется контрольная сумма, которая хранится в родительском блоке метаданных. При чтении выполняется проверка целостности данных, а при наличии резервных копий или RAID-Z конфигурации повреждённый блок может быть автоматически восстановлен. Такой подход обеспечивает защиту от деградации носителя, ошибок контроллера и других источников повреждения данных.

Несмотря на высокую эффективность подобных решений, их применение требует использования конкретной файловой системы. Это означает необходимость миграции данных, изменения структуры хранения и отказа от существующей файловой системы.

3.3. Device Mapper Integrity

В экосистеме Linux также существует подсистема Device Mapper Integrity (dm-integrity) [4], предназначенная для хранения контрольных сумм непосредственно на уровне блочного устройства. Данный механизм располагается ниже файловой системы и позволяет обнаруживать повреждения данных независимо от используемой файловой системы. На практике dm-integrity часто используется совместно с RAID и файловыми системами для создания многоуровневой защиты данных.

Однако dm-integrity в первую очередь ориентирован на контроль целостности и не предоставляет самостоятельного механизма коррекции данных без использования внешних источников избыточности, в отличие от разрабатываемого решения, которое в перспективе предполагает реализацию механизмов коррекции данных непосредственно на уровне блочного устройства без привлечения внешних подсистем хранения.

4. Коды исправления ошибок (ECC)

Коды исправления ошибок (Error Correcting Codes, ECC) [5] представляют собой класс методов помехоустойчивого кодирования, предназначенных для обнаружения и исправления ошибок в передаваемых или хранимых данных.

Основная идея ECC заключается в добавлении к пользовательским данным некоторого количества избыточной информации, вычисляемой по определённому алгоритму. При последующем чтении данных эта информация позволяет определить факт возникновения ошибки и, в ряде случаев, восстановить исходное содержимое без обращения к резервной копии данных.

Наиболее распространёнными классами кодов исправления ошибок являются:

- коды Хэмминга (Hamming Codes) [7];
- коды Рида — Соломона (Reed–Solomon Codes) [8];
- BCH-коды (Bose–Chaudhuri–Hocquenghem) [9];
- LDPC-коды (Low-Density Parity-Check) [10];
- полярные коды (Polar Codes) [11].

В рамках дальнейшего развития разработанного драйвера планируется использование кодов исправления ошибок для восстановления повреждённых блоков данных непосредственно на уровне блочного устройства.

При записи данных драйвер будет вычислять ECC-информацию и сохранять её в служебной области накопителя совместно с контрольной суммой. При обнаружении несоответствия контрольной суммы во время чтения будет выполняться попытка восстановления исходных данных на основании сохранённого ECC-кода.

Такой подход позволит не только обнаруживать повреждение данных, но и автоматически исправлять ограниченное количество ошибок без участия пользователя и без необходимости хранения полного дубликата данных.

5. Архитектура драйвера

5.1. Разбиение данных и метаданных на устройстве

В разрабатываемом драйвере существует два типа метаданных — контрольные суммы и коды коррекции ошибок. Ниже показано выбранное расположение метаданных на нижележащем блочном устройстве.



Рисунок 1 — Расположение данных и метаданных на нижележащем блочном устройстве

Как показано на Рисунке 1 весь диск делится на две части — чанкированные данные и коды коррекции ошибок. Чанкированные данные — это полезные данные и соответствующие к ним контрольные суммы, которые в паре образуют независимо обрабатываемый блок данных — чанк. В дальнейшем развитии проекта будет добавлена возможность статически определять соотношение секторов (полезных данных)/(контрольных сумм)/(ECC метаданных), а также будут проведены измерения влияния выбранных соотношений. В данный момент, чанк состоит из 9 секторов (8 секторов полезных данных и 1 сектор под контрольные суммы)

Контрольные суммы представляют собой 64 байта подряд расположенных CRC64 от каждого сектора, что позволяет достичь точности определения неисправности вплоть до сектора.

Решение хранить контрольные суммы вместе с данными исходит из того, что при каждой операции чтения и записи требуется проверять или обновлять контрольные суммы, поэтому для дисков, в которых чтение удаленных данных является более дорогой операцией, такое решение повысит производительность. При этом повреждение сектора контрольных сумм не является критической проблемой, так как драйвер будет рассчитан на восстановление полных чанков.

Компромиссом данному решению является усложнение операции ввода–вывода, как показано на Рисунке 2. Из-за чередования данных и метаданных один большой

запрос драйверу будет разбит на множество запросов нижележащему устройству — минимум по 2 запроса на каждый затронутый чанк, что требует дополнительной синхронизации и увеличивает нагрузку на планировщик ввода-вывода.

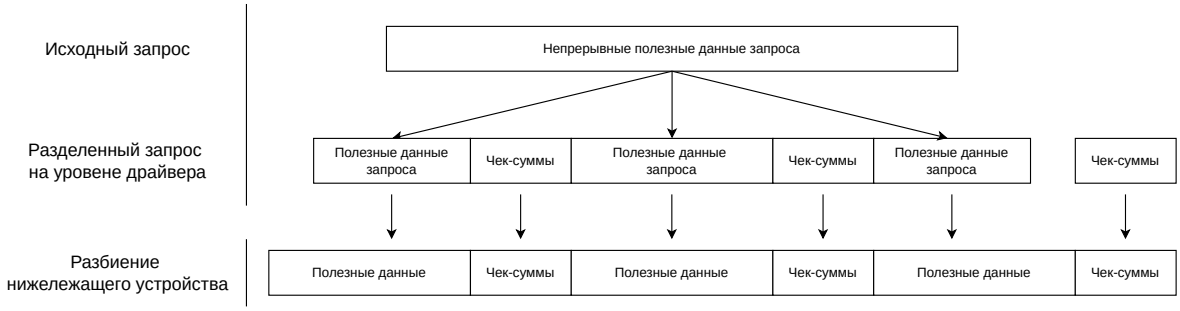


Рисунок 2 — Разбиение одного непрерывного запроса драйверу на непрерывные запросы устройству

Также стоит отметить, что из-за хранения метаданных полезный объем нижележащего устройства будет уменьшен и составит:

$$D \cdot \left\lfloor \frac{V}{D+M} \right\rfloor,$$

где D — количество полезных секторов в чанке, M — количество секторов метаданных в чанке, V — количество секторов на устройстве.

В нашем случае без учета ECC части объем составит порядка 88.8% от исходного.

5.2. Синхронизация данных и метаданных

Разрабатываемый драйвер оперирует не только полезными данными, но и метаданными (будем понимать под метаданными только контрольные суммы). Соответственно, возникает необходимость обеспечения согласованного доступа к данным и метаданным. Как было показано ранее, единицей синхронизации в разработанном драйвере является чанк, так как он является независимо–обрабатываемым блоком данных.

Для синхронизации внутренних данных чанка мы будем использовать примитив `rw_semaphore` в паре с контейнером `xarray` и ленивой инициализацией. Использование `rw_semaphore` обусловлено тем, что операции ввода-вывода могут выполняться в течение длительного времени и допускают переход потока в состояние ожидания. Кроме того, данный примитив позволяет одновременно обслуживать несколько операций чтения одного чанка, что невозможно при использовании обычного `mutex`.

Идея синхронизации крайне проста — блокировать чанк для несовместимых конкурентных операций, то есть для запись–запись и чтение–запись, при этом не хранить

ненужные данные и создавать блокировки по мере необходимости.

Одна структура блокировки в зависимости от конфигурации ядра занимает порядка 64 байт памяти. При статическом создании блокировки для каждого чанка устройства объём требуемой оперативной памяти становится неприемлемо большим. Например, для накопителя объёмом 1Tb (10^{12} байт) при размере чанка 4608 байт потребуется:

$$\frac{10^{12}}{4096 + 512} \approx 217013889 \, \frac{\text{structs}}{\text{Tb}}$$

структур блокировок, что соответствует приблизительно 13.8Gb оперативной памяти только для хранения объектов синхронизации.

Такой объём потребляемой памяти является недопустимым для системы. По этой причине в драйвере применяется ленивая инициализация блокировок. Структура синхронизации создаётся только при первом обращении к соответствующему чанку и освобождается после завершения обработки запросов. В результате объём используемой памяти определяется уровнем параллелизма рабочей нагрузки, а не размером обслуживаемого устройства.

Аналогичные принципы динамического создания объектов синхронизации используются в RAID-подсистеме ядра Linux [12] для структур синхронизации `stripe_lock`.

5.3. Поток выполнения операции записи

В данном подразделе рассматривается поток выполнения операции записи в разработанном драйвере. Обработка запроса будет асинхронной и многопоточной, затрагивающей несколько контекстов выполнения (Process, SoftIRQ).

Первым этапом обработки является проверка флагов входящего запроса, чтобы определить тип. Если это запрос на запись данных, то последовательность действий следующая (Рисунок 3):

1. Инициализируем структуру общего запроса;
2. Разделяем верхний запрос на запросы чанка с полезными данными и метаданными (Рисунок 2);
3. Инициализируем структуры запроса к чанку;
4. Инициализируем лок (но не поднимаем флаги `rw_semaphore`);

5. Инициализируем структуры для запроса к метаданным (считаем контрольные суммы запроса к чанку, создаем `bio` на запись и, если записываем не весь чанк, создаем `bio` на чтение сектора метаданных);

6. Инициализируем работу по отправке запросов у нижележащему устройству.

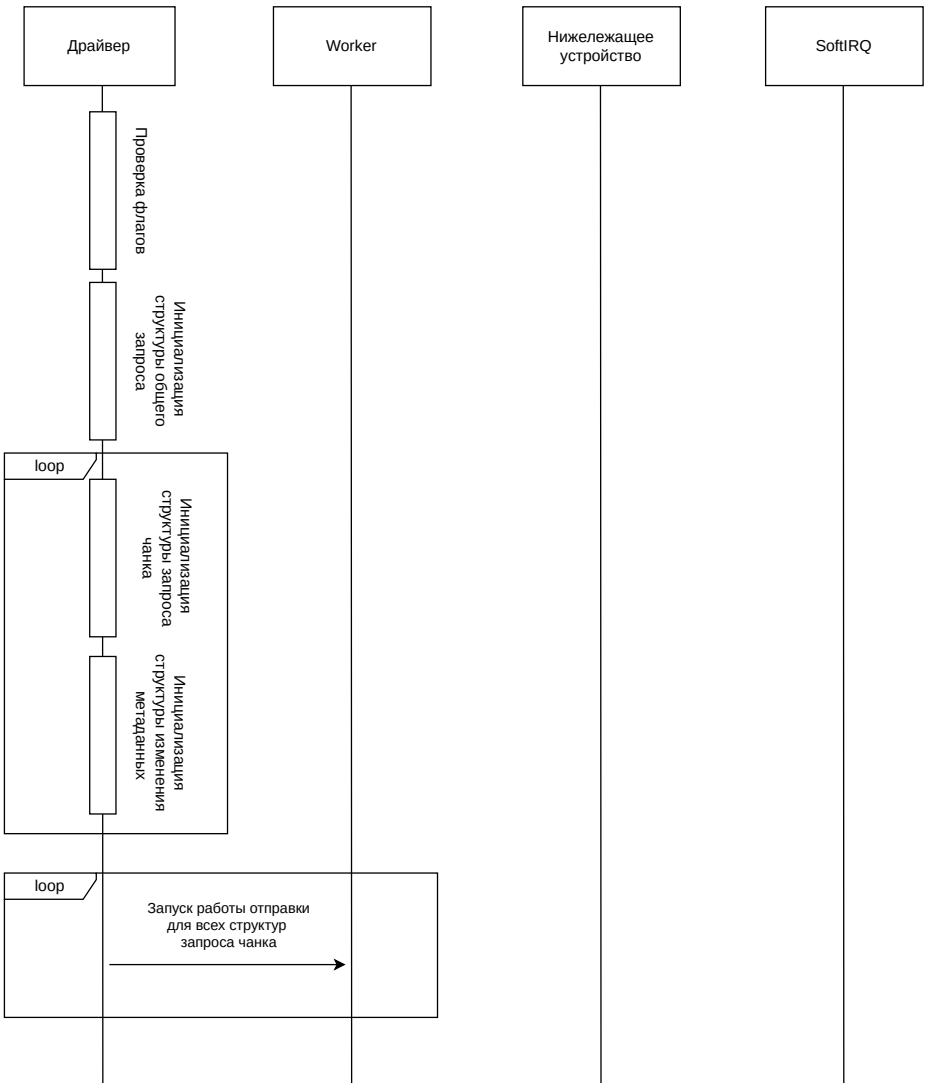


Рисунок 3 — Диаграмма последовательности обработки запроса на запись. Инициализация структур и запуск работы

Важно инициализировать все возможные структуры как можно раньше, чтобы не допускать ситуаций, когда окажется, что памяти недостаточно для всех запросов, но часть запросов уже отправлена на нижний уровень, что потребует дополнительных синхронизаций и ненужной работы, которую можно было избежать.

После завершения этапа инициализации запросы помещаются в очередь работ `work_queue`, которые выполняют следующие действия:

1. Поднятие флага лока (ожидание, если в данном чанке флаг уже поднят);
2. Отправка двух `bio` — на запись полезных метаданных и, если записываем на весь чанк, на запись метаданных, иначе на чтение метаданных.

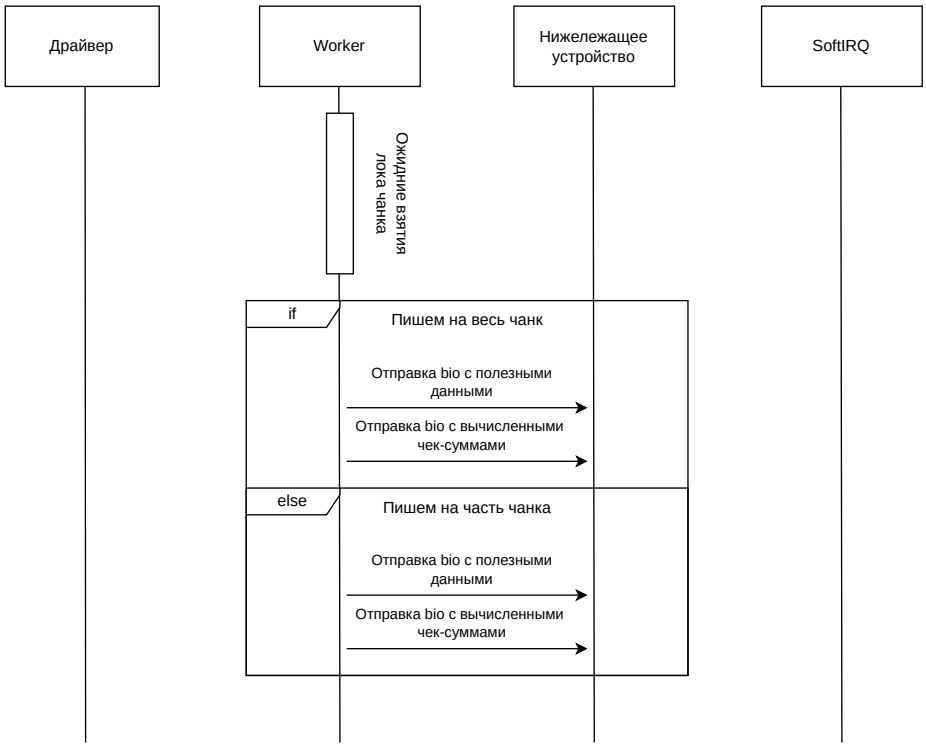


Рисунок 4 — Диаграмма последовательности обработки запроса на запись. Взятие лока и отправка запросов

При неполной записи в чанк, нам требуется обновить лишь часть контрольных сумм, но минимальной единицей записи для блочного устройства является сектор, поэтому нам требуется достать весь сектор с диска, обновить часть в оперативной памяти и отправить обновленный сектор на диск.

В данном случае мы получим не 2 запроса на чанк, а 3 (Рисунок 5), что может существенно замедлить драйвер, однако при записи крупных непрерывных диапазонов данных, большинство чанков будут перезаписываться полностью, без необходимости дополнительного чтения.

Когда нижележащее устройство закончит обработку запроса, оно вызовет SoftIRQ для завершения запроса через `bio_end_io`. В данной функции мы быстро проверяем через атомарную переменную является ли этот запрос последним запросом к чанку и, в случае положительного результата, запускаем работу по завершению запроса к чанку.

Если запрос был не на весь чанк, то значит у нас отработал запрос лишь на чтение метаданных, их требуется обновить и снова отправить запрос, но уже для записи

на диск. Дождавшись отработки и этого запроса, мы можем быть уверены, что данные и метаданные синхронизированы.

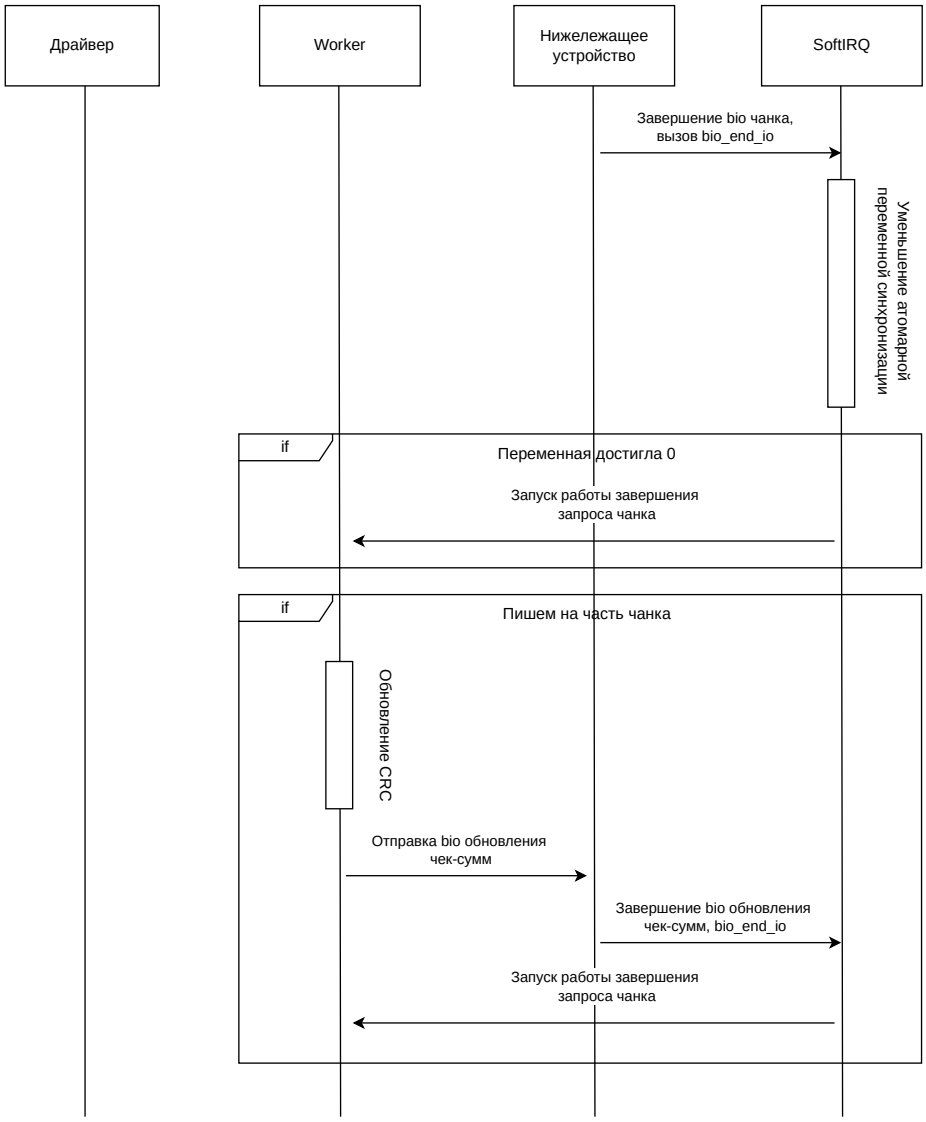


Рисунок 5 — Диаграмма последовательности обработки запроса на запись. Обработка запроса после завершения на нижележащем устройстве

Соответственно, можно освободить блокировку чанка и начать освобождать ресурсы структур связанных с запросом к чанку (Рисунок 6). Если это был последний запрос к чанку из всех созданных по запросу к драйверу (атомарная переменная синхронизации равна 0), то можем освободить ресурсы структур и завершить обработку всего запроса.

Если на каком-либо этапе произошла ошибка, то в структуре общего запроса через атомарную переменную выставляется ошибка и статус первой ошибки, который при завершении запроса передается запросу к драйверу.

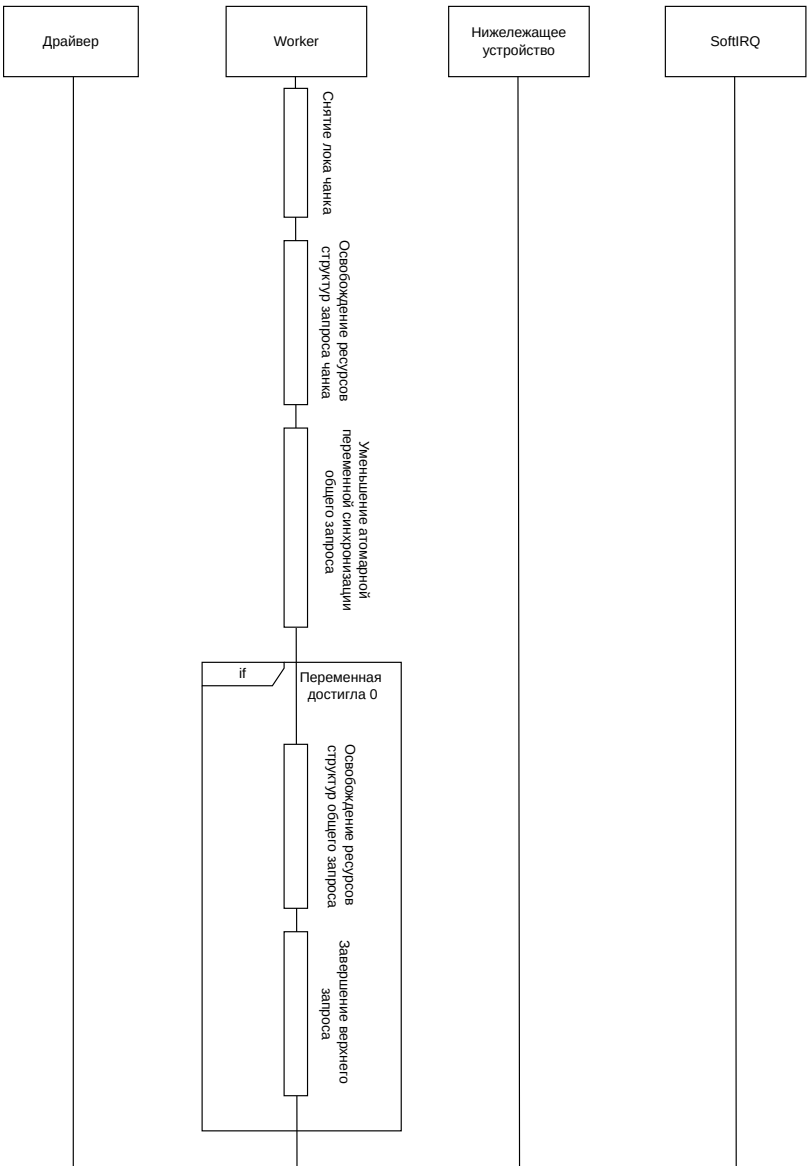


Рисунок 6 — Диаграмма последовательности обработки запроса на запись. Завершение обработки запроса к драйверу

5.4. Поток выполнения операции чтения

Поток выполнения операции чтения существенно проще потока выполнения операции записи.

Обработка начинается практически аналогично операции записи (Рисунок 3), все пункты аналогичны, кроме пункта 5, в отличие от операции записи, при чтении нам не требуется считать контрольные суммы, а также при чтении имеем ровно 2 запроса на чанк (Рисунок 7) — один на чтение полезных данных, второй на чтение метаданных.

Аналогично запросу на запись, в работе отправки запросов происходит поднятие

флагов блокировки и отправка запросов.



Рисунок 7 — Диаграмма последовательности обработки запроса на чтение. Взятие лока и отправка запросов

Также упрощается возврат после выполнения запросов нижележащим устройством (Рисунок 8), так как после возврата двух запросов чанка отправлять еще что-либо нам не потребуется.

В процедуре завершения запроса к чанку, после снятия флагов лока, лишь требуется проверить совпадение контрольных сумм полезных данных и метаданных с диска (Рисунок 9.

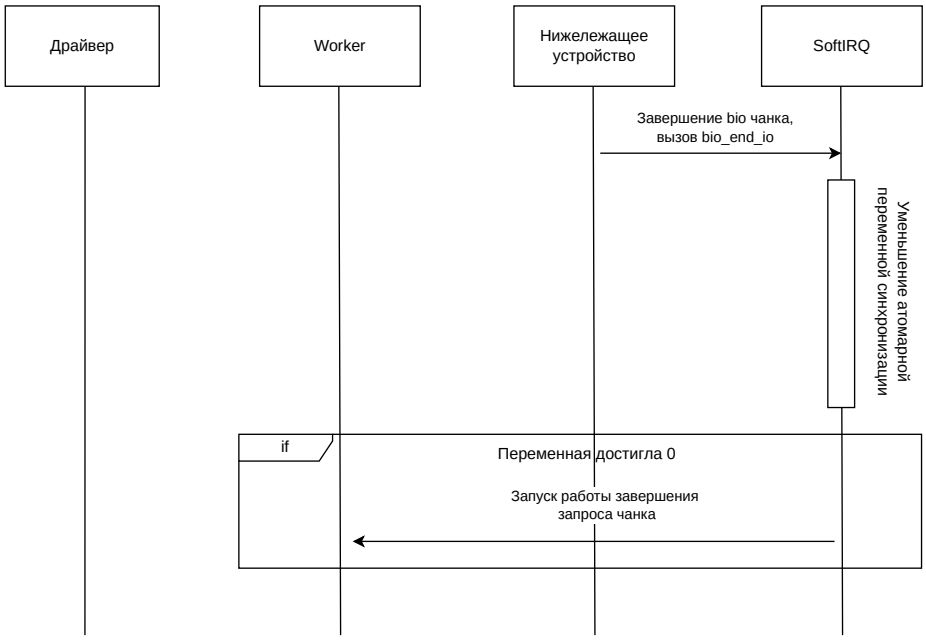


Рисунок 8 — Диаграмма последовательности обработки запроса на чтение. Обработка запроса после завершения на нижележащем устройстве

Если контрольные суммы совпадают, данные считаются корректными, после че-

го выполняется освобождение ресурсов и завершение обработки запроса, если же контрольная сумма не совпала, то, как и с запросом на чтение, атомарно выставляется ошибка и запрос к драйверу не будет успешным.

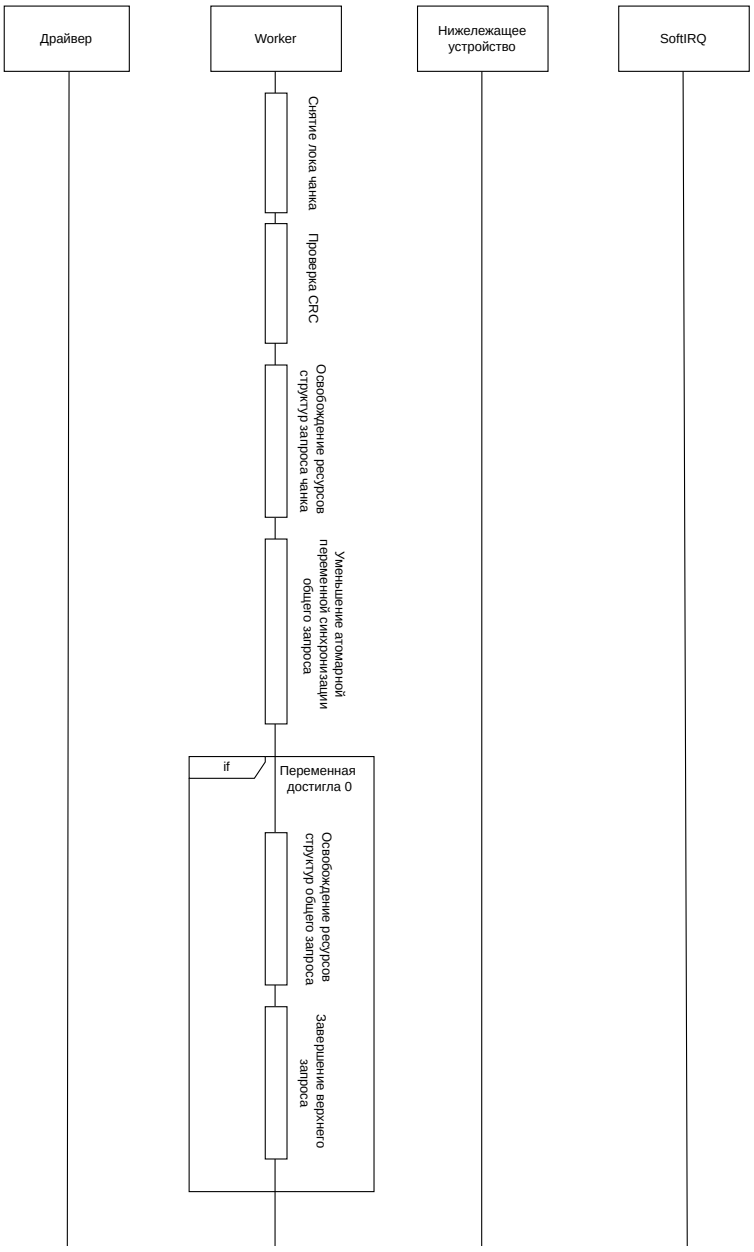


Рисунок 9 — Диаграмма последовательности обработки запроса на чтение. Завершение обработки запроса к драйверу

6. Результаты работы

6.1. Проверка корректности драйвера

Целью тестирования являлась проверка корректности функционирования разработанного драйвера блочного устройства, включая совместимость с файловыми системами, корректность выполнения операций ввода-вывода и обнаружение нарушений целостности данных.

6.1.1. Проверка корректности операций чтения и записи

Для проверки корректности обработки операций ввода-вывода использовалась утилита `fio`.

Тестирование включало последовательные и случайные операции чтения и записи с различными размерами блоков и уровнями параллелизма.

Пример используемого теста:

```
fio --name=verify \
    --filename=/dev/mapper/corrdm \
    --rw=randrw \
    --bs=4k \
    --direct=1 \
    --verify=crc32 \
    --do_verify=1 \
    --runtime=60 \
    --time_based
```

Критерием успешного прохождения теста являлось отсутствие ошибок ввода-вывода и полное совпадение записанных и прочитанных данных.

6.1.2. Проверка работы с файловой системой

Была выполнена проверка возможности создания и использования файловой системы поверх виртуального устройства, предоставляемого драйвером.

Для тестирования на устройстве была создана файловая система `ext4`:

```
mkfs.ext4 /dev/mapper/corrdm
```

После создания файловой системы устройство было смонтировано:

```
mount /dev/mapper/corrdm /mnt/test
```

В процессе тестирования выполнялись операции создания, чтения, изменения и удаления файлов различного размера.

Критерием успешного прохождения теста являлось отсутствие ошибок ввода-вывода, корректное отображение содержимого файлов и сохранение данных после команды синхронизации `sync`.

6.1.3. Проверка обнаружения повреждения данных

Для проверки механизма контроля целостности выполнялось искусственное повреждение служебных метаданных.

После записи тестовых данных определялся физический сектор устройства, содержащий контрольную сумму соответствующего блока. Затем содержимое метаданных изменялось посредством прямой записи на базовое устройство:

```
dd if=/dev/zero \
   of=/dev/vdb \
   bs=512 \
   seek=8 \
   count=1 \
   conv=notrunc
```

После повреждения метаданных выполнялось чтение сектора чанка через драйвер `/dev/mapper/corrdm`:

```
dd if=/dev/mapper/corrdm \
   of=/dev/null \
   bs=512 \
   skip=0 \
   count=1 \
   conv=notrunc
```

Драйвер повторно вычислял контрольную сумму пользовательских данных и сравнивал её со значением, сохранённым в метаданных. Поскольку контрольная информация была намеренно изменена, драйвер должен был зафиксировать нарушение целостности данных и вернуть ошибку ввода-вывода.

Успешным результатом теста считалось обнаружение повреждения, ошибка ввода–вывода и регистрация соответствующего сообщения в журнале ядра Linux.

6.1.4. Итоги проверки корректности

Анализ результатов тестирования показал, что разработанное решение удовлетворяет большинству функциональных требований, сформулированных в главе «Постановка задачи». Степень покрытия функциональных требований составляет около 90%. Таким образом, текущая реализация обеспечивает полный цикл обнаружения нарушений целостности данных и создаёт необходимую основу для дальнейшего развития системы коррекции ошибок.

Исходный скрипт проверки корректности доступен вместе с исходным кодом драйвера [13].

6.2. Тесты производительности

6.2.1. Методика проведения тестов производительности

Для оценки влияния механизмов контроля целостности данных на производительность подсистемы хранения было выполнено сравнение разработанного драйвера с исходным блочным устройством без дополнительной обработки данных.

Измерения проводились при помощи утилиты `fio`, предназначенной для генерации синтетических нагрузок на подсистему ввода-вывода. В качестве исследуемых характеристик рассматривались количество операций ввода-вывода в секунду (IOPS), пропускная способность и задержка выполнения операций.

Для оценки производительности были выбраны четыре типовых сценария работы:

- последовательное чтение блоками 512 байт;
- последовательная запись блоками 512 байт;
- последовательное чтение блоками 16 KiB;
- последовательная запись блоками 16 KiB.

Размер блока 512 байт соответствует минимальному размеру логического сектора большинства накопителей и позволяет оценить влияние накладных расходов драйвера

при работе с большим количеством мелких запросов. Размер блока 16 KiB был выбран как более характерный для современных файловых систем и приложений, работающих с относительно крупными объёмами данных.

Во всех экспериментах использовались прямые операции ввода-вывода (`O_DIRECT`), что исключало влияние файлового кэша операционной системы на результаты измерений. Для создания конкурентной нагрузки на подсистему хранения использовались глубина очереди запросов 32 и 8 параллельных потоков выполнения.

Каждый тест выполнялся десять раз. По результатам измерений вычислялось среднее значение исследуемой характеристики. Для оценки разброса результатов дополнительно рассчитывалась стандартная ошибка среднего (Standard Error, SE), которая отображается на графиках в виде доверительных интервалов.

Аппаратная среда выполнения:

- Процессор — Intel(R) Core(TM) i7-8700K (12) @ 4.70 GHz;
- Оперативная память — 2x Corsair Vengeance LPX DDR4 16Gb;
- Накопитель — Samsung SSD 980 1TB;
- Интерфейс подключения — PCI Express 3.0 x4.

Программная среда выполнения (QEMU/KVM):

- Операционная система — Fedora Linux 42 (Server Edition) x86_64;
- Ядро Linux — 6.19.11-100.fc42.x86_64;
- Версия fio — 3.37;

Все тесты производительности драйвера доступны вместе с исходным кодом [13].

6.2.2. Результаты тестов производительности

На рисунке 10 представлено сравнение количества операций ввода-вывода, выполняемых исходным устройством и разработанным драйвером при неконкурентном, однопоточном доступе.

Как и ожидалось, использование дополнительного уровня обработки данных приводит к снижению количества операций ввода-вывода в секунду. Интересным оказалось поведение нижележащего устройства, которое, при операции чтения, не теряет

количества операций при увеличении размера запросов, а при операции записи даже увеличивает свою скорость обработки. Скорее всего это связано с тем, что SSD накопители оперируют большими блоками данных, соответственно, запрос на 16KiB более удобен контроллеру и может требовать меньше операций.

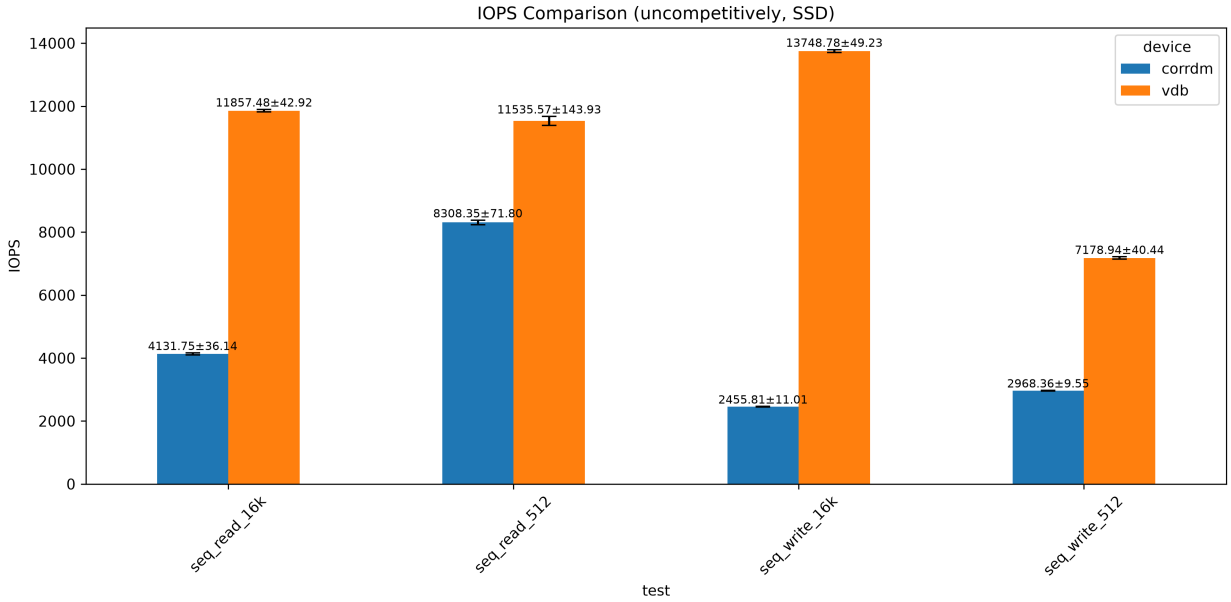


Рисунок 10 — Сравнение количества операций ввода-вывода драйвера и устройства. Однопоточный тест.

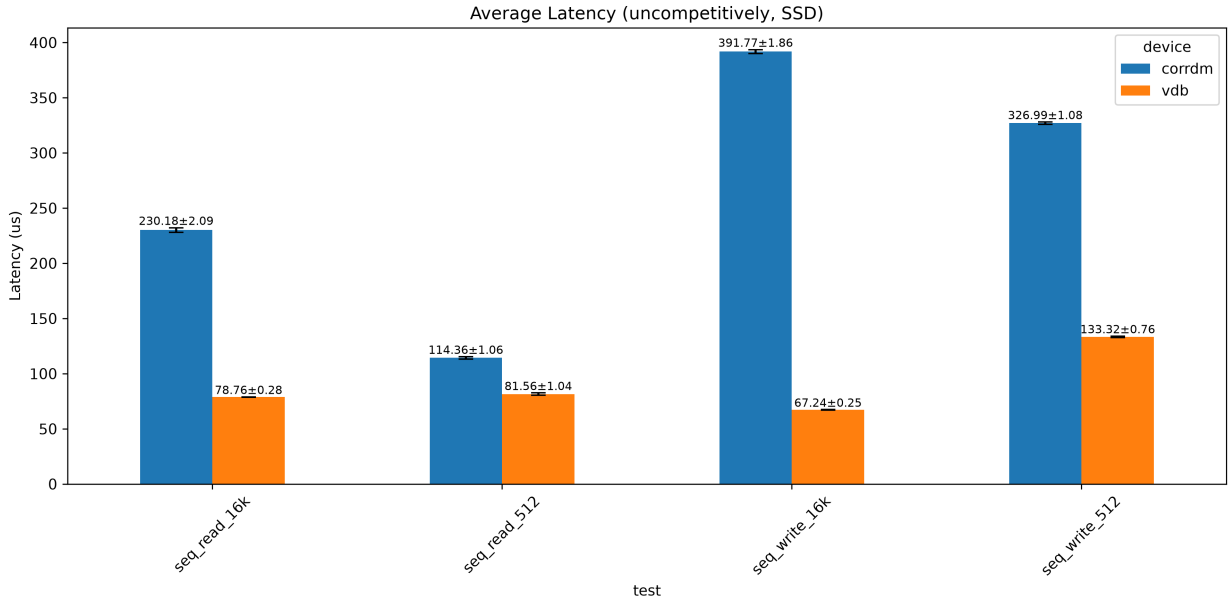


Рисунок 11 — Сравнение средней задержки операций ввода-вывода драйвера и устройства. Однопоточный тест.

Полученные значения относительного снижения производительности по количеству операций ввода-вывода в секунду представлены ниже:

- Чтение блоками 16 KiB — 2.86 ± 0.02;

- Чтение блоками 512 — 1.39 ± 0.02;
- Запись блоками 16 KiB — 5.60 ± 0.03;
- Запись блоками 512 — 2.42 ± 0.02;

Аналогичные соотношения видны на Рисунке 11 и на Рисунке 12.

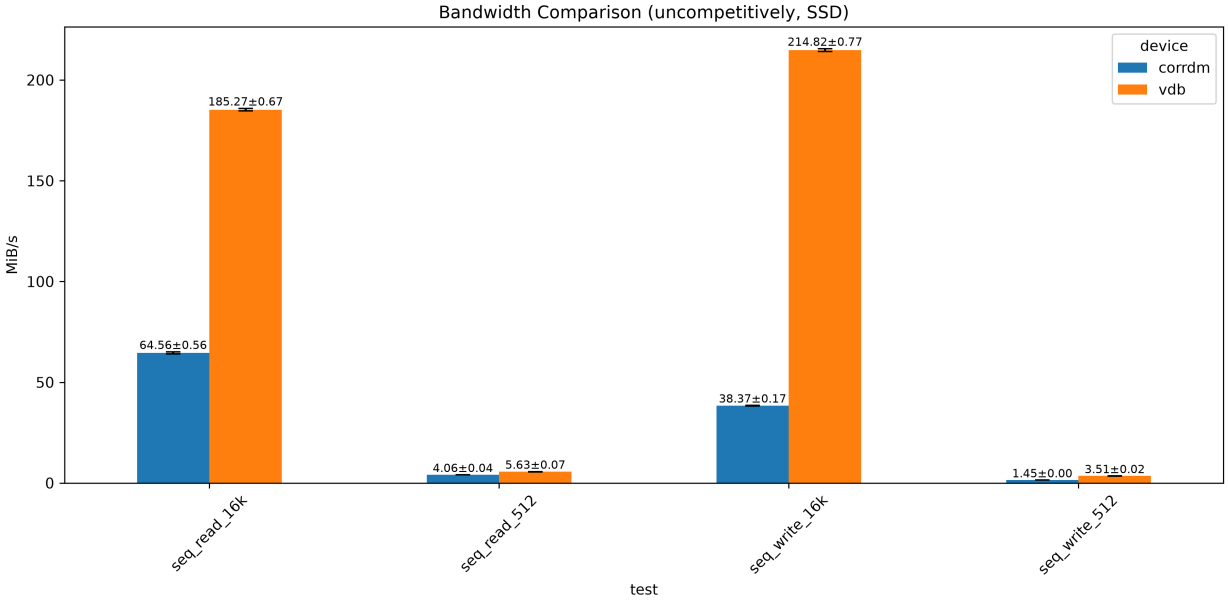


Рисунок 12 — Сравнение пропускной способности драйвера и устройства. Однопоточный тест.

Для объяснения наблюдаемого снижения производительности необходимо оценить вклад непосредственно драйвера в общее время обработки запросов. С помощью систем дебага ядра `/sys/kernel/debug/tracing` можно увидеть среднее время выполнения функций драйвера в Таблице 1 и в Таблице 2:

Функция	**Количество вызовов**	**Общее время (μs)**	**Среднее время (μs)**
dm_map	164029	1436335.880	8.757
trn_rq_init	164029	1004804.895	6.126
submit_work	656116	1907668.300	2.908
complete_trn_rq	164029	447047.740	2.725
complete_work	656116	1764856.100	2.690
trn_rq_submit	164029	396355.113	2.416
trn_p_rq_init	656116	835371.800	1.273
trn_mr_rq_init	656116	462739.305	0.705
check_crc	656116	448380.700	0.683
trn_p_rq_end_io	1312232	788015.050	0.601
complete_trn_mr_rq	656116	290836.590	0.443
locker_put_lock	656116	257975.260	0.393
locker_get_lock	656116	219403.747	0.334
metadata_bio_init	656116	88598.935	0.135

Таблица 1. Профилирование функций и время выполнения для однопоточного теста чтения блоками 16 KiB.

Функция	**Количество вызовов**	**Общее время (μs)**	**Среднее время (μs)**
dm_map	72390	853847.450	11.795
trn_rq_init	72390	634219.215	8.761
complete_trn_rq	72390	237042.380	3.275
submit_work	289738	878795.210	3.033
trn_rq_submit	72390	204359.009	2.823
complete_work	289738	614196.910	2.120
trn_p_rq_init	289738	552647.246	1.907
trn_mw_rq_init	289200	379183.103	1.311
trn_p_rq_end_io	579476	392736.040	0.678
bio_crc_calc.isra.0	289200	147325.347	0.509
complete_trn_mw_rq	289200	145350.434	0.503
locker_put_lock	289738	116133.801	0.401
locker_get_lock	289738	104439.177	0.360
metadata_bio_init	289738	38924.858	0.134

Таблица 2. Профилирование функций и время выполнения для однопоточного теста записи блоками 16 KiB.

Как видно из результатов профилирования, время выполнения отдельных функций драйвера измеряется единицами микросекунд и само по себе не способно объяснить наблюдаемое многократное снижение производительности относительно исходного устройства. Следовательно, основным источником дополнительных затрат является увеличение количества запросов к нижележащему устройству. В зависимости от расположения данных и метаданных один пользовательский запрос может порождать несколько операций ввода–вывода, что приводит к дополнительным накладным расходам на обработку и ожидание завершения операций, то есть, это следствие разбиения пространства устройства на чанки.

Теперь рассмотрим многопоточный тест, результаты которого видны на Рисунках 13, 14, 15.

Соотношения производительности на каждом рисунке почти аналогичны и составляют:

- Чтение блоками 16 KiB — 2.83 ± 0.11;
- Чтение блоками 512 — 7.85 ± 0.09;
- Запись блоками 16 KiB — 4.70 ± 0.27;
- Запись блоками 512 — 32.14 ± 0.21;

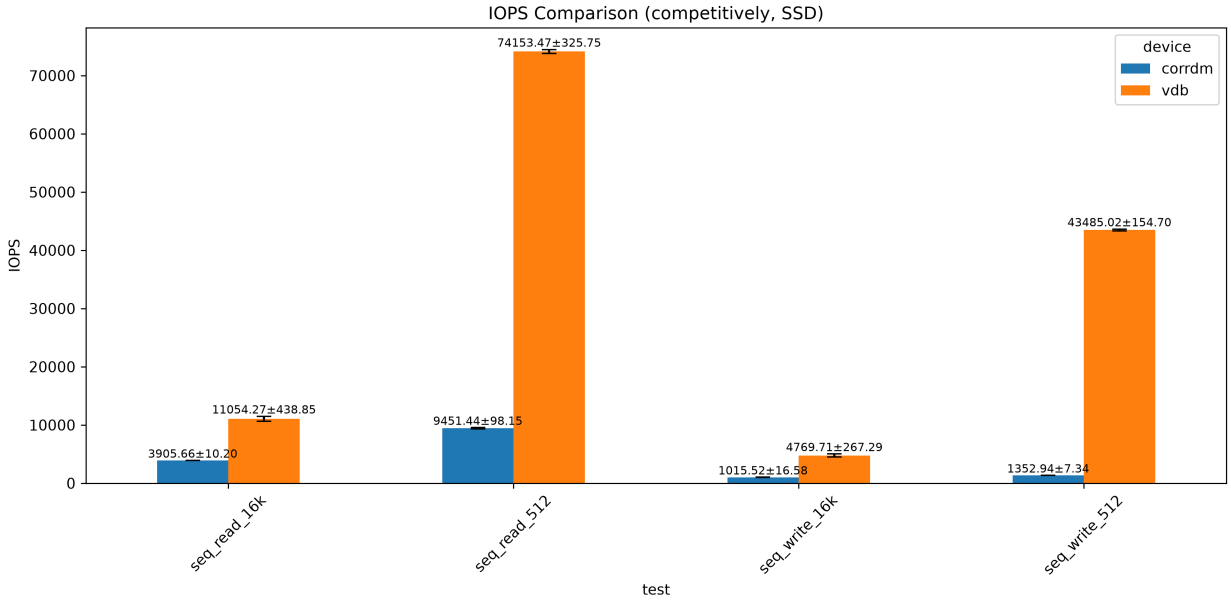


Рисунок 13 — Сравнение количества операций ввода-вывода драйвера и устройства. Многопоточный тест.

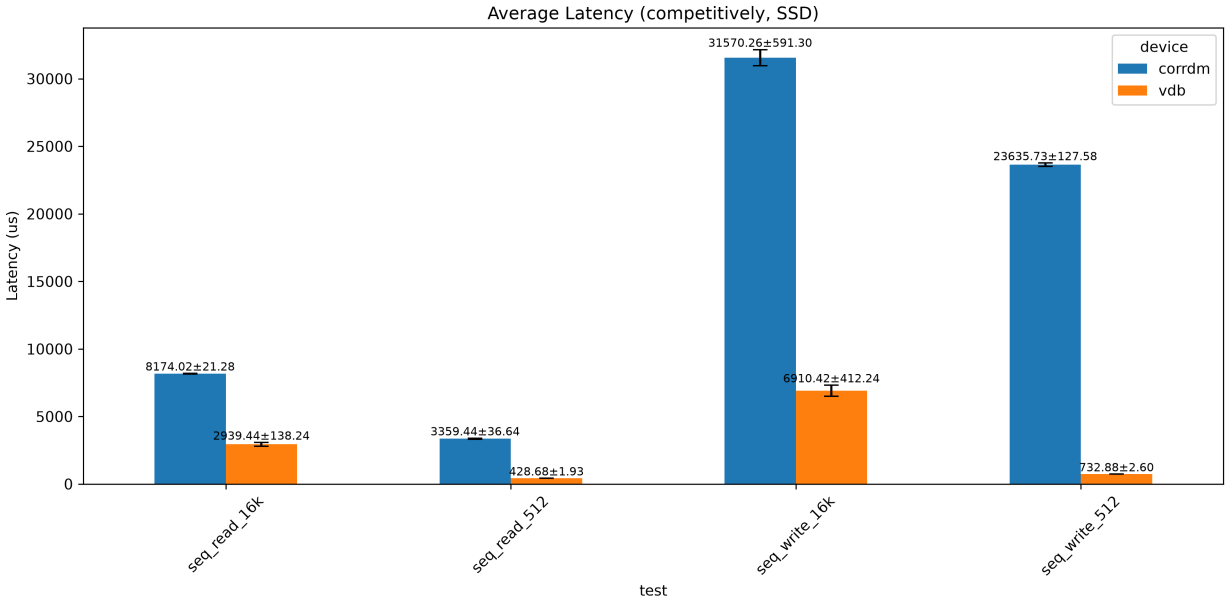


Рисунок 14 — Сравнение средней задержки операций ввода-вывода драйвера и устройства. Многопоточный тест.

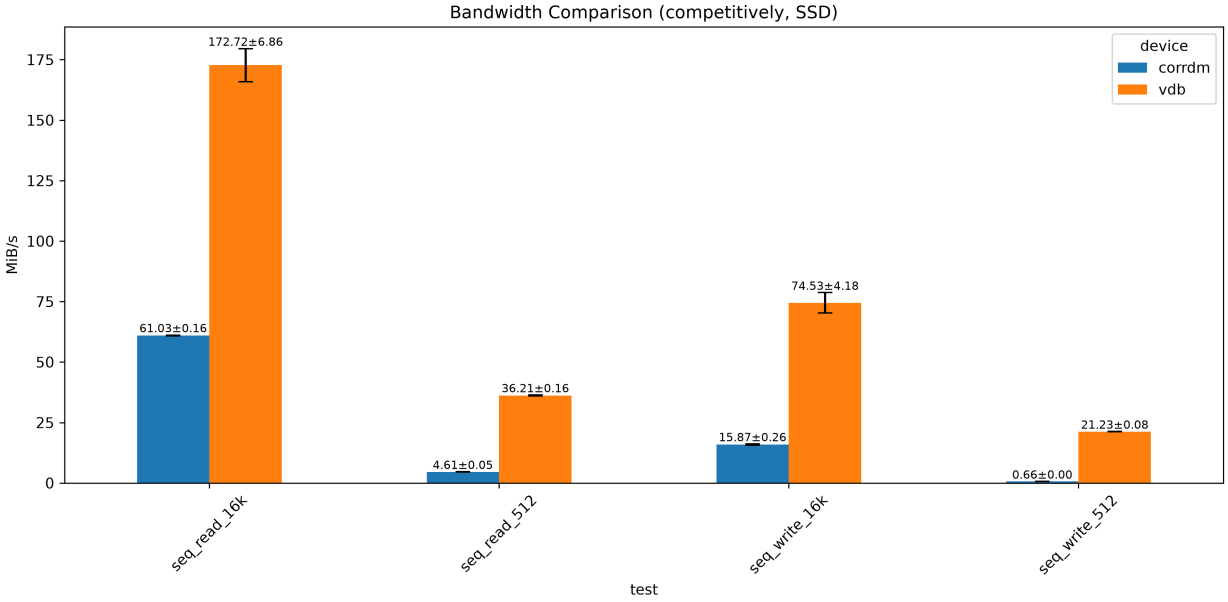


Рисунок 15 — Сравнение пропускной способности драйвера и устройства. Многопоточный тест.

Существенное снижение производительности при записи блоками размером 512 байт может быть объяснено особенностями выбранной схемы хранения данных и метаданных, а именно тем, что драйверу гарантированно требуется по 3 операции к нижележащему устройству, что в свою очередь увеличивает время эксклюзивной блокировки чанка. Данное предположение подтверждается результатами профилирования, представленными в Таблице 3 и Таблице 4.

Результаты профилирования показывают, что функция `submit_work` становится

основным узким местом разработанного решения. Именно на данном этапе выполняется ожидание доступа к чанку, что приводит к существенному росту времени обработки запросов в условиях конкурентной нагрузки, которое может объяснить разницу на Рисунке 14.

Функция	**Количество вызовов**	**Общее время (μs)**	**Среднее время (μs)**
submit_work	3826408	366255867.000	95.718
dm_map	956602	13823950.000	14.451
trn_rq_init	956602	7775300.700	8.128
trn_rq_submit	956602	5783104.700	6.045
complete_work	3826408	13600325.000	3.554
complete_trn_rq	956602	2750363.700	2.875
trn_p_rq_init	3826408	6364585.700	1.663
check_crc	3826408	4411841.600	1.153
trn_mr_rq_init	3826408	3349960.200	0.875
trn_p_rq_end_io	7652816	5395726.300	0.705
locker_put_lock	3826408	2139196.900	0.559
complete_trn_mr_rq	3826408	1913081.100	0.500
locker_get_lock	3826408	1719726.500	0.449
metadata_bio_init	3826408	773256.850	0.202

Таблица 3. Профилирование функций и время выполнения для многопоточного теста чтения блоками 16 KiB.

Функция	**Количество вызовов**	**Общее время (μs)**	**Среднее время (μs)**
submit_work	996960	27993735963.000	28079.096
dm_map	249151	4812636.900	19.316
trn_rq_init	249151	2996694.400	12.028
trn_rq_submit	249151	1744903.500	7.003
complete_trn_rq	249151	1161011.900	4.660
complete_work	996960	3037188.600	3.046
trn_p_rq_init	996960	2589077.400	2.597
trn_mw_rq_init	995884	1796320.800	1.804
trn_p_rq_end_io	1993920	2723595.600	1.366
bio_crc_calc.isra.0	995884	795797.250	0.799
complete_trn_mw_rq	995884	618743.270	0.621
locker_put_lock	996960	551439.210	0.553
locker_get_lock	996960	449331.050	0.451
metadata_bio_init	996960	209956.730	0.211

Таблица 4. Профилирование функций и время выполнения для многопоточного теста записи блоками 16 KiB.

Полученные результаты демонстрируют зависимость производительности драйвера от размера обрабатываемых запросов. При увеличении размера блока наблюдается рост пропускной способности. Это объясняется уменьшением относительных накладных расходов на обработку каждого пользовательского запроса, а также более эффективными блокировками чанков.

Наибольшее снижение производительности наблюдается при многопоточной записи небольшими блоками данных. Анализ профилирования показал, что в данном режиме основным ограничивающим фактором становится механизм синхронизации доступа к чанкам.

Следует отметить, что вычисление и проверка контрольных сумм не являются основным источником снижения производительности. Согласно результатам профилирования, соответствующие функции занимают незначительную долю общего времени обработки запросов. Основные накладные расходы обусловлены необходимостью координации доступа к данным и метаданным при конкурентном использовании, а также организацией дополнительных операций ввода-вывода.

Несмотря на выявленное снижение производительности по сравнению с исходным устройством, полученные результаты соответствуют ожидаемым для выбранной архитектуры. Реализованные механизмы контроля целостности требуют выполнения дополнительной работы, связанной с обработкой метаданных, синхронизацией параллельных запросов и поддержанием корректности данных. Данные затраты являются неизбежной платой за обеспечение дополнительного уровня надёжности хранения информации.

Вместе с тем проведённое исследование позволило выявить ряд направлений для дальнейшей оптимизации, которые описаны в следующем параграфе.

6.3. Предложения по улучшению архитектуры

Проведённый анализ производительности позволил выявить основные ограничения текущей архитектуры драйвера и определить направления её дальнейшего совершенствования.

Наиболее существенной проблемой является необходимость удержания блокировки чанка на протяжении выполнения операций ввода-вывода. В существующей реализации блокировка защищает согласованность данных и метаданных, однако её длительное удержание приводит к существенному снижению производительности при конкурентном доступе.

Одним из возможных способов решения данной проблемы является внедрение внутреннего кэша драйвера. В этом случае операции чтения, модификации и формирования метаданных могут выполняться в оперативной памяти без удержания блокировки на время обращения к накопителю. Блокировка потребуется только на этапе син-

хронизации локального состояния с устройством хранения, что позволит значительно сократить время её удержания и повысить степень параллелизма обработки запросов.

Другим направлением оптимизации является уменьшение количества операций ввода-вывода, необходимых для обработки одного пользовательского запроса. В текущей реализации один внешний запрос может порождать несколько обращений к нижележащему устройству для чтения или записи данных и связанных с ними метаданных. Использование буферизации метаданных, объединения запросов или более компактного размещения служебной информации может снизить количество дополнительных операций и уменьшить накладные расходы.

Таким образом, несмотря на успешную реализацию базового механизма контроля целостности данных, текущая архитектура обладает значительным потенциалом для дальнейшей оптимизации.

7. Заключение

В ходе выполнения научно-исследовательской работы был разработан драйвер блочного устройства для операционной системы Linux на основе подсистемы Device Mapper, обеспечивающий контроль целостности данных на уровне блочного ввода-вывода.

В процессе работы были проанализированы существующие подходы к обеспечению надёжности хранения данных, включая RAID-массивы, файловые системы Btrfs и ZFS, а также подсистему dm-integrity. На основании проведённого анализа была предложена собственная архитектура решения, ориентированная на независимость от используемой файловой системы и возможность контроля целостности данных непосредственно на уровне блочного устройства.

В разработанном драйвере реализованы механизмы хранения служебных метаданных, вычисления и проверки контрольных сумм, преобразования адресного пространства, а также синхронизации конкурентного доступа к данным и метаданным. Особое внимание было уделено обеспечению согласованности данных при параллельном выполнении операций чтения и записи.

Проведённое функциональное тестирование подтвердило корректность работы драйвера. Разработанное решение успешно выполняет операции чтения и записи, корректно функционирует совместно с файловой системой ext4 и обнаруживает нарушения целостности данных при искусственном повреждении метаданных.

Результаты тестов производительности показали, что реализация механизмов контроля целостности приводит к снижению производительности по сравнению с исходным блочным устройством. Полученные результаты соответствуют ожидаемым ограничениям выбранной архитектуры и обусловлены дополнительными операциями обработки метаданных, вычисления контрольных сумм и синхронизации доступа к чанкам данных.

Таким образом, поставленная цель работы была достигнута. Разработан и протестирован драйвер блочного устройства, обеспечивающий контроль целостности данных на уровне блочного ввода-вывода и создающий основу для дальнейшего развития проекта.

Естественные продолжением дальнейших исследований является реализация механизмов автоматической коррекции ошибок на основе кодов исправления ошибок (ECC), а также оптимизация архитектуры драйвера с целью уменьшения накладных расходов и повышения производительности при обработке запросов ввода-вывода.

8. Список использованных источников

[1] Derek Vadala. Managing RAID on Linux. O'Reilly Media, 2003.

[2] Btrfs Documentation. https://btrfs.readthedocs.io/en/stable/

[3] OpenZFS Documentation. https://openzfs.github.io/openzfs-docs/

[4] Milan Brož, Mikuláš Patóčka, Vashek Matyáš. Practical Cryptographic Data Integrity Protection with Full Disk Encryption (2018).

[5] Lin S., Costello D. Error Control Coding: Fundamentals and Applications. Pearson, 2004.

[6] Eduardo Pinheiro, Wolf-Dietrich Weber and Luiz Andre Barroso. Failure Trends in a Large Disk Drive Population, 2007.

[7] Richard W. Hamming, Error Detecting and Error Correcting Codes, Bell System Technical Journal, 1950.

[8] Irving S. Reed, Gustave Solomon, Polynomial Codes Over Certain Finite Fields, 1960.

[9] Raj Chandra Bose, Dwijendra Kumar Ray-Chaudhuri, On A Class of Error Correcting Binary Group Codes, 1960.

[10] Robert G. Gallager, Low-Density Parity-Check Codes, 1962.

[11] Erdal Arıkan, Channel Polarization: A Method for Constructing Capacity-Achieving Codes for Symmetric Binary-Input Memoryless Channels, 2009.

[12] RAID source code:
https://elixir.bootlin.com/linux/latest/source/drivers/md/dm-raid.c

[13] Исходный код драйвера:
https://github.com/Dx-by-Dy/Automatic-Error-Correction-Driver